

Midterm review

Lectures 1–8, in eight pages and fourteen problems

Fall 2026 · Week 7 · the midterm is in class, Thu 10/22

How to use this

This is a review, not new material. Everything here has appeared in a lecture, an assignment, or an earlier recitation.

Work in this order, and do not reverse it. **First** read Section 1, which is the whole of Lectures 1–8 compressed into the statements you should be able to make without notes. **Then** do the problems in Section 2 with the summary closed. Reading a summary produces the feeling of understanding; answering a question produces the fact of it, and the gap between those two is where midterm marks are lost.

What the exam can ask. Every lecture in this course does three things: introduces an *object* (a distance, a matrix, a rule), states *what it is for*, and states *when it misleads you*. Exam questions come from all three, and the third is the one students prepare least and are asked about most.

1 Lectures 1–8, compressed

1.1 L01 — Representations, spaces, and metrics

A representation is a pattern that stands for something. Put n stimuli down the rows and d units across the columns and you have the **response matrix** X , which is the object the first half of this course operates on.

The **dot product** $a \cdot b = \sum_i a_i b_i = \|a\| \|b\| \cos \theta$ is a measurement of b along the direction a .

Three distances, and why there are three:

Distance	Sensitive to	Blind to
Euclidean	magnitude and pattern	nothing
cosine	pattern only	overall scale
correlation	pattern after centering	scale <i>and</i> additive offset

Correlation distance is cosine distance on row-centered vectors. It is *undefined* for a constant row — a dead unit — because that row has zero length after centering.

The **metric axioms**: (i) $d(a, b) = 0$ iff $a = b$ (minimality); (ii) $d(a, b) = d(b, a)$ (symmetry); (iii) $d(a, c) \leq d(a, b) + d(b, c)$ (the triangle inequality). Cosine and correlation distance violate the triangle inequality. Human similarity judgments violate all three: people rate “North Korea is like China” higher than the reverse, and identical twins are not judged maximally similar to themselves.

1.2 L02 — Similarity, MDS, and clustering

A **dissimilarity matrix** (RDM) is $n \times n$, symmetric, zero diagonal. It discards the units and keeps only the geometry, which is why 50,000 voxels and 4,096 network units become directly comparable.

MDS finds coordinates whose distances reproduce a given dissimilarity matrix. Metric MDS fits the distances; non-metric MDS fits only their *rank order*, which is the weaker and often more defensible assumption for behavioral data. Three warnings when reading an MDS map: the axes have no inherent meaning, the whole map can be rotated or reflected freely, and a 2-D solution may be a poor summary of the matrix that produced it.

Shepard's universal law: perceived similarity falls exponentially with distance in psychological space, $s = a e^{-bd}$. It is a law about *psychological* space, so applying it to raw network features is not a test of it.

Hierarchical clustering always returns a tree, whether or not the data has one. Choosing where to cut chooses a level of description, not a true number of clusters. The linkage rule (single, complete, average) changes the answer, so it must be reported.

Comparing two RDMs: Spearman correlation on the upper triangle, `np.triu_indices(n, k=1)`. The pairs are not independent, so the accompanying p -value is not trustworthy.

1.3 L03 — PCA and eigendecomposition

An image is a vector of pixels. High-dimensional spaces are mostly empty and distances in them lose contrast — the **curse of dimensionality** — but real data occupies a far smaller subspace than it has room for, which is what makes dimensionality reduction possible at all.

Center the data, form the covariance $C = \frac{1}{n-1} \tilde{X}^\top \tilde{X}$, take its eigenvectors. Each eigenvector is a direction C stretches without rotating ($Cv = \lambda v$); each eigenvalue says how much variance lies along it. PC1 is the direction of greatest spread. Components are orthogonal and ordered.

Projection gives coordinates in the new basis; **reconstruction** with K components is the best rank- K approximation in squared error. Eigenfaces are exactly this on face images.

Three cautions: PCA is **linear**, so a curved manifold is not captured; it is sensitive to **scale**, so features in different units should be standardized first; and **centering is not optional**, since without it PC1 usually just points at the mean.

1.4 L04 — t -SNE and UMAP

t -SNE converts distances to neighbor probabilities in high dimensions, builds a heavy-tailed version in two dimensions, and moves the points to match the two distributions. **Perplexity** is roughly the effective number of neighbors each point tries to keep. The heavy tail exists to solve the **crowding problem**: two dimensions do not have enough room for all the moderate distances that exist in high dimensions.

How to misread it, in three ways that all appear in published figures: cluster sizes mean nothing, distances *between* clusters usually mean nothing, and apparent clusters can appear in data with none. One picture is never the result — vary the seed and the perplexity.

UMAP is faster, tends to preserve more global structure, and shares all the same warnings.

1.5 L05 — The geometry of neural representations

A stimulus condition is not a point but a **manifold**: repeated presentations, viewpoints, and nuisance variation trace out a cloud.

Effective dimensionality summarizes the eigenspectrum in one number — for instance the participation ratio $(\sum_i \lambda_i)^2 / \sum_i \lambda_i^2$. It is *estimated*, not observed, and it depends on the stimulus set and the number of samples as much as on the system.

Mixed selectivity: units that respond to combinations of variables raise the dimensionality of the population code and make more categorizations linearly separable. The cost is abstraction — high-dimensional codes generalize less readily. That trade-off is the lecture's core claim.

The **metric zoo** — RSA, CCA, CKA, shape metrics — exists because two systems have no shared coordinates. The metrics disagree with one another, which is a live problem, not a detail.

1.6 L06 — The model neuron

$a = g(\mathbf{w} \cdot \mathbf{x} + b)$. The weights are a direction, so a neuron **measures how much of its preferred pattern is present**; the bias shifts the threshold; g is the nonlinearity, usually ReLU.

Because $\mathbf{w} \cdot \mathbf{x} = \|\mathbf{w}\| \|\mathbf{x}\| \cos \theta$, a strong response has two possible causes — an intense stimulus or a well-matched one — and separating them requires normalizing the input. Among inputs of fixed length, the best stimulus for a linear unit is its own weight vector.

A **tuning curve** is response against a stimulus parameter. A single linear-threshold unit computes AND, OR, and NOT, but not XOR.

1.7 L07 — How one neuron learns

Rule	Update	Needs a teacher?	Converges to
Hebb	$\Delta \mathbf{w} = \eta a \mathbf{x}$	no	PC1's direction, but $\ \mathbf{w}\ $ diverges
Oja	$\Delta \mathbf{w} = \eta a (\mathbf{x} - a \mathbf{w})$	no	PC1, unit length
Perceptron	$\Delta \mathbf{w} = \eta (y - \hat{y}) \mathbf{x}$, on errors only	yes	some separating boundary
Delta	$\Delta \mathbf{w} = \eta (y - \hat{y}) \mathbf{x}$, every example	yes	the least-squares solution

Hebbian learning finds PC1 because the average weight change is $C\mathbf{w}$, and repeatedly applying C amplifies the top eigenvector fastest. **Centering matters**: on uncentered data the rule chases the mean instead. Oja's rule adds one decay term that normalizes the weight, and that is the entire difference.

The perceptron and the delta rule share an update but not a behavior: the perceptron updates only on mistakes and stops at the first boundary that separates, so *which* boundary you get depends on the order of presentation. The delta rule updates always, minimizes squared error, and converges to one answer regardless of order. The perceptron convergence theorem bounds the number of updates by $(R/\gamma)^2$, where γ is the margin — so a small margin means many updates, and non-separable data means it never stops.

Rescorla-Wagner is the delta rule in animal learning: $\Delta V_i = \eta(\lambda - \sum_j V_j)$. Because the error uses the *sum* of all cues' predictions, a cue already predicted by another cue learns nothing — which is **blocking**, and it is why the rule was a landmark.

1.8 L08 — Multilayer networks and backpropagation

Stacking linear layers gives a linear map, so **the nonlinearity is not optional**. XOR is solvable by a 2-2-1 network: the hidden layer re-represents the four points so that they become linearly separable, which is the whole idea of a hidden layer in one example.

Universal approximation: one hidden layer of sufficient width can approximate any continuous function on a bounded domain to any accuracy. It says nothing about how wide, and nothing about whether gradient descent can *find* the weights. Depth is about efficiency, not possibility.

Backpropagation is the chain rule applied backwards through the layers, reusing each layer's stored quantities. It solves credit assignment. It is not an optimizer — gradient descent is; backprop only supplies the gradient.

Batch (all data per step), minibatch (a subset), stochastic (one example). The learning rate is the main knob: too small is slow, too large oscillates or diverges.

2 Problems

Do these closed-book. Answers follow.

1. A unit's activity is (2, 4, 6) across three stimuli; another's is (1, 2, 3). Give the cosine distance and the correlation distance between them, and explain the pair of answers in one sentence.
2. Why is correlation distance undefined for a unit that produces the same value for every stimulus?
3. You are told $d(A, B) = 0.2$, $d(B, C) = 0.2$, $d(A, C) = 0.9$. Which axiom fails, and name a distance used in this course that permits it.
4. Someone reports that an MDS map's horizontal axis is "animacy". What is the methodological objection, and what evidence would answer it?
5. You forget to center your data before PCA on a set of images. Describe PC1.
6. A representation gives 90% of variance in 8 components. The stimulus set is 40 objects at 30 viewpoints. Why is "this representation is 8-dimensional" unsafe?

7. Two t -SNE runs on identical data, differing only in `random_state`, give visibly different maps. Is either wrong? What should be reported?
8. State the trade-off that mixed selectivity buys and pays for.
9. A linear unit has $w = (3, 4)$ and $b = 0$. Give its response to $x = (1, 0)$, and to the unit-length input that maximizes its response.
10. Hebb's rule is run on data with a large positive mean. What does the weight converge toward, and what one preprocessing step fixes it?
11. Write Oja's rule and say precisely what the extra term relative to Hebb's rule accomplishes.
12. The perceptron is trained twice on the same separable data, differing only in the order of examples, and gives two different boundaries. Explain, and say whether the delta rule would do the same.
13. In Rescorla–Wagner, cue A is trained to predict a reward for 50 trials. Then A and B are presented together for 50 more, still rewarded. What happens to V_B , and what is this phenomenon called?
14. A 2–2–1 network fails to learn XOR from some initializations. Give one concrete mechanism, and one diagnostic you could run on a failed network.

Answers

1. Cosine distance 0: the two vectors point in exactly the same direction, differing only by a factor of two. Correlation distance 0 as well: after centering each row, both become proportional to $(-1, 0, 1)$. The pair of answers says these units carry the same information about the stimuli, differing only in gain — which is why neither distance can tell them apart, and why Euclidean distance would.
2. Centering a constant row gives the zero vector, whose norm is zero, and the correlation is a division by that norm. In code it surfaces as `nan`, which then propagates silently through the whole matrix.
3. The triangle inequality: $0.9 > 0.2 + 0.2$. Cosine distance and correlation distance both permit this.
4. MDS axes carry no inherent meaning — the solution can be rotated or reflected freely, so “the horizontal axis” is not a property of the data. Answering it requires independent evidence: correlate the coordinate with an external animacy rating, or show that a classifier trained on that coordinate alone predicts animacy on held-out stimuli.
5. PC1 points approximately at the mean image, because the uncentered second-moment matrix is dominated by it. You have spent your first component describing the average, and the interesting structure moves to PC2 onward.
6. The 1,200 images come from only 40 distinct objects; the 30 viewpoints of one object trace a smooth low-dimensional curve rather than adding independent directions. The figure is partly a property of how the stimulus set was built and would move if you used 10 viewpoints with an unchanged system.

7. Neither is wrong; both are members of a family generated by the algorithm's random initialization. Report what is stable across seeds (and across perplexities), and say how many you ran.
8. It buys **separability**: with units tuned to combinations, more categorizations become linearly readable, and the effective dimensionality rises. It pays in **abstraction**: a high-dimensional code generalizes less readily to new conditions, because it encodes conjunctions rather than factors.
9. $\mathbf{w} \cdot (1, 0) = 3$. The maximizing unit-length input is $\mathbf{w}/\|\mathbf{w}\| = (0.6, 0.8)$, giving a response of $\|\mathbf{w}\| = 5$.
10. Toward the mean direction of the data rather than the direction of greatest *variance* — Hebbian dynamics follow the second-moment matrix, which equals the covariance only after centering. Subtract the mean.
11. $\Delta \mathbf{w} = \eta a (\mathbf{x} - a \mathbf{w})$. The $-\eta a^2 \mathbf{w}$ term is a decay proportional to the unit's own activity: it grows exactly when the weight grows, driving $\|\mathbf{w}\|$ toward 1 and preventing the unbounded growth of plain Hebbian learning, without changing the direction it converges to.
12. The perceptron updates only when it makes a mistake, and stops as soon as no mistakes remain — so it halts at whichever separating boundary it reaches first, and which one that is depends on the order the mistakes arrived in. The delta rule minimizes a squared error with a single minimum and would converge to the same boundary regardless of order.
13. V_B stays near zero. The prediction error is $\lambda - (V_A + V_B)$ and V_A already accounts for the reward, so there is no error left to drive learning about B. This is **blocking**. (Remove phase 1 and both cues learn, sharing the association.)
14. Mechanism: a hidden unit's pre-activation is negative for all four inputs, so its ReLU gradient is zero and it never updates — a dead unit, leaving the network effectively 2-1-1, which cannot solve XOR. Diagnostic: run the four inputs through the trained network and count, for each hidden unit, how many inputs it responds to with a non-zero value. A unit answering zero for all four is dead. (Also acceptable: both hidden units converging to the same function, or a saddle point.)

The night before

Six things worth being able to state cold, because each has appeared in a lecture, an assignment, and a recitation:

1. the three distances and what each is blind to;
2. the three metric axioms, and one real violation of each;
3. what an eigenvector of the covariance is, in a sentence with no algebra;
4. why Hebb needs centering and what Oja's extra term does;
5. why the perceptron's answer depends on presentation order and the delta rule's does not;

6. why a stack of linear layers is a linear layer.